

# Solutions for Practice Problems for Week 4

Operating and Systems Programming 2022/23

1. Threads make it possible to execute several parts of the program code concurrently. This is particularly useful for servers, which can create a separate thread for each incoming request.
2. The following are always shared:
  - Global variables
  - Data in the heap
  - Open files

Data in stacks is shared if the stack frames were created before the thread has been started.

3. A critical section is a piece of code where switching between threads must be limited to ensure correct execution of the program.
4. Other threads may have to wait and cannot do any useful work while one thread executes a critical section. You want to keep this waiting time as short as possible.
5. Other threads might have to wait for a very long time or not run at all if the thread is blocked forever.
6. The critical sections are only the lines `totalCalls++`, as all other variables which are manipulated by the two threads are not shared.
7. You will need a standard lock around both critical sections. A read-write lock cannot be used, as both threads modify the shared variable.
8. The critical sections are the whole of `fun1` and all instructions of `fun2` including the variable `msg`. Read-write-locks may be used, as `fun1` modifies the pointer `msg`, but `fun2` only reads the value of the pointer `msg`. To make sure the critical sections are as short as possible, the code of `fun2` needs to be re-organised. The code should look like:

```
#include <stdio.h>
#include <stdlib.h>
```

```

char *msg = "This is a message";
pthread_rwlock_t lock = PTHREAD_RWLOCK_INITIALIZER;

void fun1 () {

    pthread_rwlock_wrlock(&lock);
    if (msg != NULL) {
msg = NULL;
    }
    pthread_unlock(&lock);
}

char *fun2 () {
    char *result;

    result = malloc(6);
    pthread_rwlock_wrlock(&lock);
    if (msg == NULL || (strlen(msg) > 5)) {
        pthread_unlock(&lock);
        free(result);
return NULL;
    }
    strcpy(result, msg);
    pthread_unlock(&lock);
    return result;
}

```